



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	Programming from First Principles	
Topic Name	Block Structure 1.1	
Topic Code	-	
Unit/Module No. & Name	1	Standard Constructs
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-5
2.0	Meaning And Definition	6-8
3.0	Nature Or Type	9-10
4.0	Examples And Case Studies	11-15
5.0	Keywords And Touch Vocabulary	16-17



Savitribai Phule Pune University
Centre For Distance and Online Education

OBJECTIVE

1. This unit explains the basic block structure of PL/SQL.
2. The topic is also known as Block Structure 1.1 in MCA courses.
3. It is a core skill for database programming.
4. A PL/SQL block is a small and complete program unit.
5. Every block has a clear start and a clear end.
6. The code inside a block follows a fixed order.
7. Learning block structure helps in writing clean and safe database logic.
8. Proper structure makes programs easy to read and test.
9. After this unit, students can identify the three parts of a PL/SQL block.
10. Students learn which block parts are mandatory and which are optional.
11. Students can write an anonymous PL/SQL block.
12. They can declare variables inside a block.
13. They can run simple SQL statements in PL/SQL.
14. They can print output for learning and debugging.
15. They can handle common runtime errors using exceptions.
16. Students understand scope, meaning where variables can be used.
17. Scope helps avoid confusion in large programs.
18. The concept is linked to real-life campus examples.
19. Examples include fee payment systems, result grading, banking transactions, and library fine calculation.
20. The content is written in simple English for first-time learners.

1.0 INTRODUCTION

Databases store important information. In a college system, the database stores student details, course registrations, marks, fee payments, library records, and login data. SQL is the main language used to communicate with a database. Using SQL, we can select data, insert new rows, update existing rows, and delete rows. SQL is very powerful, but it is mainly a query language. It works best when a task can be completed using a single statement.

In real life, many tasks cannot be done in one SQL statement. For example, a college portal may need to check a rule, calculate a value, save a record, and then display a message. A banking system may need to verify an account balance, create a transaction record, update the account, and handle errors safely. A result-processing system may need to read many student records, decide grades, and update the database. These tasks need multiple steps, decisions, and loops, and they may need temporary values during execution.

PL/SQL is a programming language that works together with SQL to handle step-by-step tasks. PL/SQL stands for Procedural Language extensions to SQL. The word “procedural” means that the language supports ordered steps, conditions, and loops. PL/SQL adds variables, decision statements, loops, and error handling around SQL. This allows database programs to run close to the data and apply business rules correctly and consistently.

One important feature of PL/SQL is its block structure. A block is a small program unit designed to perform one task. It contains its own variables and statements. A block has a clear start, a clear execution phase, and a clear end. It can also catch and handle errors in a controlled way. Errors can happen due to missing data, wrong data types, division by zero, or permission problems. Without proper error handling, a program may stop suddenly and confuse the user.

Beginners usually learn PL/SQL using lab tools such as Oracle SQL Developer, SQL*Plus, or online practice tools. In these tools, PL/SQL is written as a script. Each statement ends with a semicolon, and a slash (/) on a new line tells the tool to execute the block. During practice, students often use DBMS_OUTPUT.PUT_LINE to display output. This output is mainly for learning and debugging. In real applications, results are usually returned to the application layer instead of being printed on the screen.

A common beginner mistake is mixing SQL rules with PL/SQL rules. In SQL, the equals sign (=) is used for comparison in WHERE clauses, and beginners may think it is also used for assignment. In PL/SQL, the correct assignment operator is :=. Another common mistake is forgetting the INTO clause when using SELECT in PL/SQL. In PL/SQL, SELECT must store its result into variables using INTO. If the query returns no row, the NO_DATA_FOUND exception occurs. If it returns more than one row, the TOO_MANY_ROWS exception occurs. These rules help ensure controlled and predictable results.

The block structure helps in clear thinking and clean coding. It clearly shows where variables are declared, where actions are performed, and where errors are handled. This reduces confusion and makes debugging easier. It also helps teachers evaluate student programs because the code follows a standard format.

For Online MCA students, block structure is important in both theory and practice. In theory exams, students are often asked about the parts of a block, their correct order, and the difference between anonymous and named blocks. In practical exams, students are usually asked to write a short anonymous block that uses variables, SELECT INTO, or exception handling. Later topics such as procedures, functions, triggers, and packages also use the same block concept, so this topic forms a strong foundation.

A simple way to understand a block is to imagine it as a room with three areas. The first area is for planning, where variables are declared. The second area is for doing the work, where SQL statements and calculations are written. The third area is for handling problems, where errors are managed. When the block ends, the room closes and the temporary variables disappear.

A good study method is to memorize the block skeleton first: DECLARE, BEGIN, EXCEPTION, END. Then students can slowly add one variable, one SQL statement, and one output line. After that, they can add one IF condition and one loop. This step-by-step practice reduces fear and builds confidence.

In exams, questions often test simple but important points, such as the correct order of block sections, mandatory and optional parts, and the assignment operator. Questions may also ask why SELECT INTO is required and what happens when a query returns no rows or multiple rows. In practical exams, common tasks include reading one row into variables, updating a row, and handling NO_DATA_FOUND. A good answer shows clear indentation, correct semicolons, and a well-defined exception section.

In projects, good practice is to keep PL/SQL blocks small and focused. Each block should solve one clear problem, such as calculating a fine or storing a payment. If a block becomes too long, it should be divided into smaller procedures. Errors should not be hidden. WHEN OTHERS should be used carefully to display messages and log error details such as SQLCODE and SQLERRM. This improves maintenance and accountability in real systems.

Finally, students should test their blocks using normal data and edge cases. Normal data is expected input, such as a positive amount. Edge cases include zero values, negative values, missing rows, and duplicate rows. Testing these cases helps students understand the importance of exception handling. For online learners, running the same block multiple times and changing one line at a time is an effective way to learn faster and avoid mistakes.

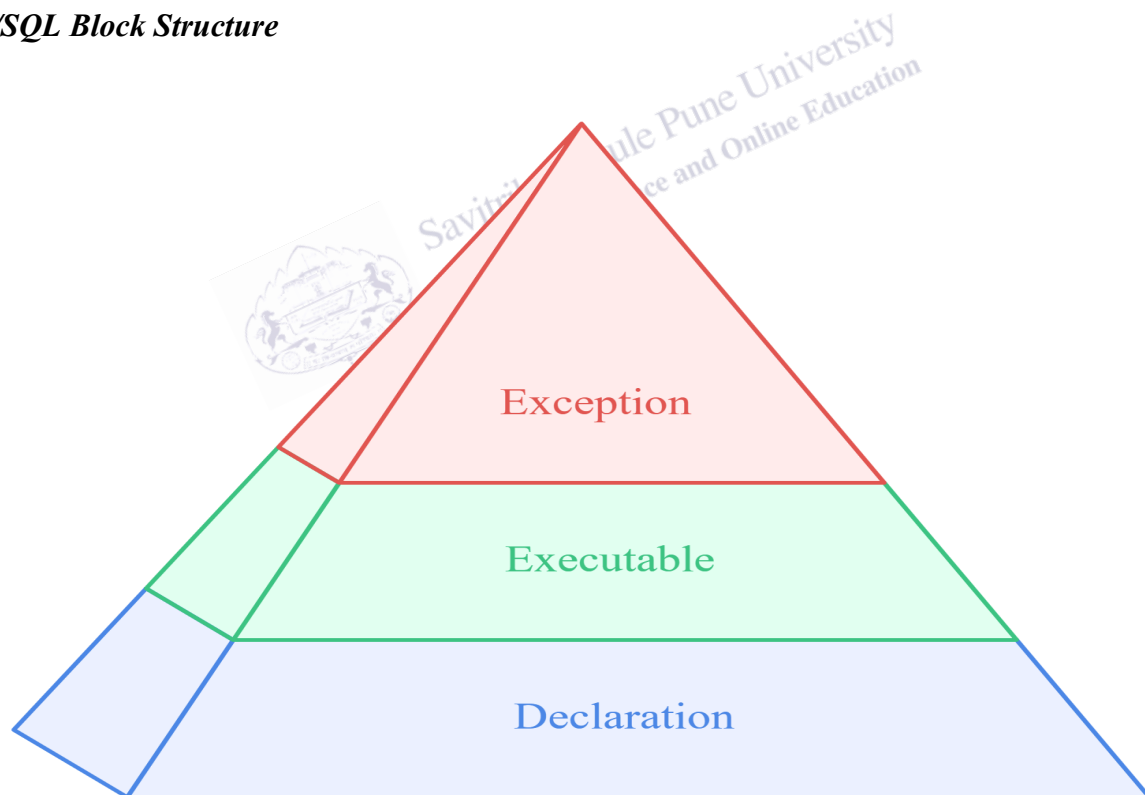
2.0 MEANING AND DEFINITION

A PL/SQL block is a structured unit of PL/SQL code that is compiled and executed as one whole program unit. In simple terms, it is a complete mini program with a defined boundary. The boundary is important because it tells the database where the program starts and where it ends. In PL/SQL, the block ends with the keyword END followed by a semicolon. In many tools, a slash on a new line is used to ask the tool to submit the entire block to the database engine.

A standard PL/SQL block can contain three sections, written in a fixed order. First, the declaration section begins with the keyword DECLARE. This section is optional. It is used to define identifiers that the block will use while it runs. Identifiers include variables, constants, cursors, records, and user-defined exceptions. Declaring variables before use improves clarity and prevents many errors. Each variable is given a data type, such as NUMBER, VARCHAR2, or DATE.

Fig. 1

PL/SQL Block Structure



The image illustrates the PL/SQL block structure, showing three main sections: Declaration for variables, Executable for program logic, and Exception for handling runtime errors in a structured and controlled way.

Second, the executable section begins with the keyword `BEGIN`. This section is mandatory. It contains statements that perform the task. These statements can include SQL statements and PL/SQL statements. SQL statements include `SELECT`, `INSERT`, `UPDATE`, and `DELETE`. PL/SQL statements include assignment using `:=`, decision making using `IF`, repetition using `LOOP`, and procedure calls. In most programs, the `BEGIN` section is the longest part because it contains the main logic.

Third, the exception section begins with the keyword `EXCEPTION`. This section is optional. It contains handlers for errors that happen during execution. A handler begins with `WHEN`, names the exception, and then lists steps to take when that error occurs. For example, the handler for `NO_DATA_FOUND` can display a message or insert a row into an error log table. A general handler, `WHEN OTHERS`, can catch any error, but it should be used carefully so serious problems are not hidden without a trace.

The block structure also supports readable documentation. In academic and professional work, code should be understandable. Clear names like `v_student_id` and `v_amount` help the reader understand purpose. Comments can also be used. A single-line comment starts with two hyphens, and a multi-line comment is written between `/*` and `*/`. Comments should explain why a rule exists, not repeat obvious code. For example, “reject negative fee amount” is useful, while “this is an IF statement” is not helpful.

Another part of meaning is controlled entry and exit. A block starts at `BEGIN` and ends at `END`. This means the block has one entry point and one exit point. Control statements may change the path inside, but the block boundary stays the same. This discipline is useful for debugging, because a learner can print messages at key points and confirm program flow. It is also useful for review, because the whole unit can be tested as one object.

Blocks can declare constants. A constant is declared using the keyword `CONSTANT` and must be assigned a value. Constants help when a value should not change, such as a fixed tax rate in a sample problem. Blocks can also declare user-defined exceptions. A user-defined exception is declared in the `DECLARE` section and raised in the `BEGIN` section using `RAISE`, or using `RAISE_APPLICATION_ERROR` for an error number and message. This supports clean rule enforcement because the program can stop when a rule is broken and show a clear message.

The meaning of a block also includes scope. Scope is the region of code where an identifier is visible. If a variable is declared in a block, it is visible inside that block. When the block finishes, that variable no longer exists. If there is an inner block inside an outer block, the inner block can use variables from the outer block. However, the outer block cannot use variables declared only inside the inner block. This rule prevents accidental use of temporary values and reduces name conflicts.

Blocks can also be classified by how they are used. An anonymous block is a block without a name. It is not stored in the database. It is written and run directly in a tool. Anonymous blocks are common for learning, testing, one-time scripts, and quick checks. A named block is stored and given a name. Procedures and functions are named blocks that can be called by applications and users. Triggers are named blocks that run automatically when a database event happens. Packages are named containers that store related subprograms together. Even though these program units have extra syntax, their core logic still uses the same block structure.

A key academic definition can be stated as follows: “A PL/SQL block is the fundamental syntactic and logical unit of PL/SQL, consisting of an optional declarative part, a mandatory executable part, and an optional exception-handling part.” This definition shows structure, order, and purpose.



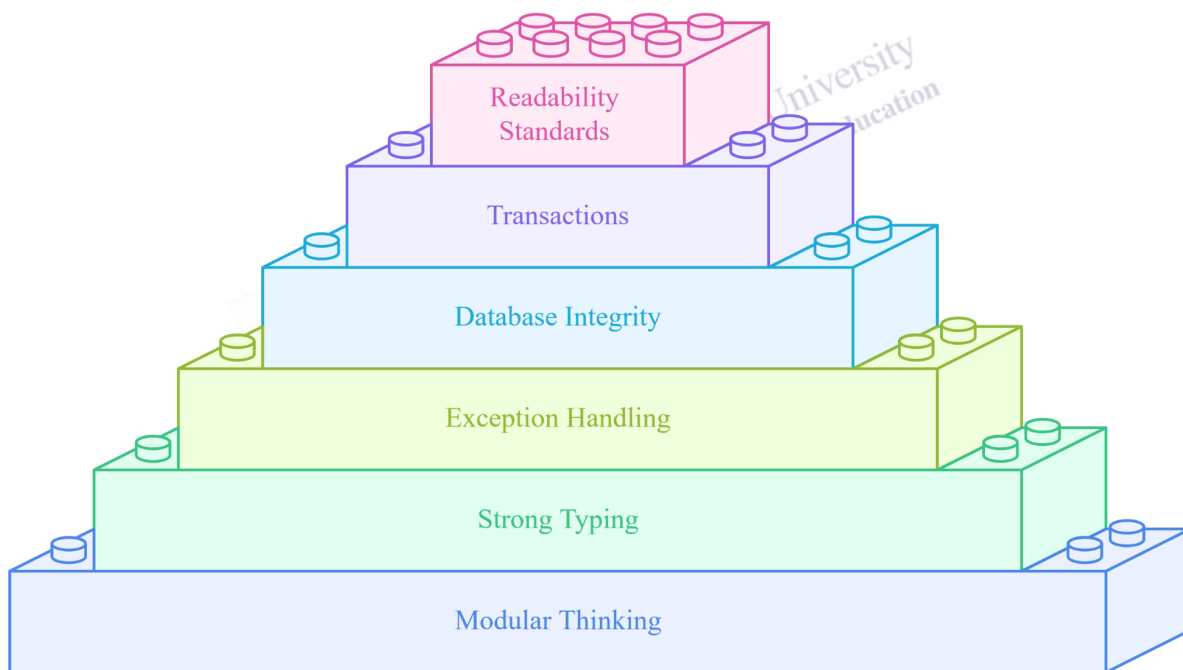
3.0 NATURE OR TYPE

The block structure in PL/SQL shows the nature of PL/SQL as a disciplined language for database work. The structure is designed to separate duties. Declarations are separated from actions, and actions are separated from error responses. This separation makes programs more reliable, especially when they work with critical data.

One nature of block structure is modular thinking. A module is a small part of a larger system. Each module can be tested and understood. A block is a module. A large application may call many stored procedures. Each procedure may contain several blocks. When each block has a clear job, the system becomes easier to maintain.

Fig. 2

PL/SQL Block Structure Pyramid



The image shows the PL/SQL block structure pyramid, highlighting modular thinking, strong typing, exception handling, database integrity, transactions, and readability standards as layered principles for writing clear, safe, and reliable PL/SQL programs.

Another nature is strong typing. In PL/SQL, variables have declared data types. A block encourages a programmer to plan types in the DECLARE section. This reduces runtime errors

and makes code clearer. For example, a variable declared as DATE shows it holds a date. A variable declared with %TYPE shows it matches a table column type. This is useful because if the table column type changes later, the variable type can still match without manual edits.

Blocks can be described by type in several ways. By storage, blocks can be transient or stored. Anonymous blocks are transient because they are not saved inside the database. Named blocks are stored and can be shared by many users. By execution trigger, blocks can be executed directly or executed automatically. A procedure runs when it is called, while a trigger runs automatically when an event happens. By nesting, blocks can be simple or nested. Nesting is useful when a small part of logic needs its own exception handling, or when a small unit needs its own local variables.

Exception handling has important behavior rules. When an exception occurs in an inner block, PL/SQL first looks for a matching handler inside that inner block. If no handler matches, the exception propagates to the outer block. This means the error moves outward until a handler is found or the program ends. This behavior allows a programmer to catch a small error locally without stopping a larger process. It also allows broader errors to be handled at a higher level, where logging and rollback decisions can be taken.

For example, an outer block may loop through many student records. Inside the loop, an inner block may compute a scholarship amount for one student. If the inner block fails due to missing data for that one student, the inner block can handle the error and continue the loop. Without such structure, one missing value could stop the entire batch and delay results for everyone.

The block structure also interacts with database integrity rules. Tables may have primary keys, foreign keys, not null rules, and check constraints. When a block tries to insert or update data, these rules can reject invalid values. The resulting errors can be handled in the exception section with clear messages and safe rollback. In academic writing, it is useful to note that integrity rules are a strong first defense, and PL/SQL validations are an additional layer that can provide user-friendly feedback.

Transactions are another part of nature. A transaction is a set of changes treated as one unit. In PL/SQL, a block can make many changes. The developer must decide where to commit or rollback. In many applications, commits are handled by the application, while PL/SQL focuses on correct logic. In scripts and batch jobs, commits may be included in the block. The key idea is that a well-designed block should not leave the database half updated.

Finally, the block structure supports readability standards. A well-formatted block uses indentation and helpful naming. Indentation makes it easy to see nested IF and LOOP statements. Naming makes it clear what each value means. For undergraduate learners, these habits reduce errors and improve long-term project quality.

4.0 EXAMPLES AND CASE STUDIES

Examples make the block structure easier to remember. Each example below shows the same core idea: a block has a start, a set of steps, and an end. Some examples also show how errors are handled.

Example 1: A minimal anonymous block. This block does not declare variables. It prints a message.

```
BEGIN
DBMS_OUTPUT.PUT_LINE('PL/SQL block started and ended safely. ');
END;
/
```

This example shows that DECLARE is optional, but BEGIN and END are required. It also shows a common method for output during learning.

Example 2: Declaring a variable and assigning a value. The assignment operator in PL/SQL is :=, not =.

```
DECLARE
v_semester NUMBER := 1;
BEGIN
DBMS_OUTPUT.PUT_LINE('Semester: ' || v_semester);
END;
/
```

Example 3: Using SELECT INTO with exception handling. Assume a table STUDENT with STUDENT_ID and STUDENT_NAME. The block reads a name and handles missing data.

```
DECLARE
v_name STUDENT.STUDENT_NAME%TYPE;
BEGIN
SELECT student_name INTO v_name
FROM student
WHERE student_id = 101;

DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_name);
EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('No student found for ID 101.');
```

```
END;  
/
```

Example 4: A nested block with local error handling. This pattern is useful when one step is risky, but the overall task should continue.

```
DECLARE  
v_total NUMBER := 100;  
BEGIN  
  BEGIN  
    v_total := v_total / 0;  
  EXCEPTION  
    WHEN ZERO_DIVIDE THEN  
      DBMS_OUTPUT.PUT_LINE('Inner block handled divide by zero.');  END;  
  
  DBMS_OUTPUT.PUT_LINE('Outer block continues with total: ' || v_total);  
END;  
/
```

Case Study 1: Fee payment validation. A fee module should not accept a negative amount. It should store the payment only when valid.

```
DECLARE  
v_student_id NUMBER := 101;  
v_amount NUMBER := 1500;  
BEGIN  
  IF v_amount <= 0 THEN  
    RAISE_APPLICATION_ERROR(-20001, 'Invalid payment amount.');  END IF;  
  
  INSERT INTO payment(student_id, amount, paid_on)  
  VALUES (v_student_id, v_amount, SYSDATE);  
  
  DBMS_OUTPUT.PUT_LINE('Payment stored for student ' || v_student_id);  
EXCEPTION  
  WHEN OTHERS THEN  
    DBMS_OUTPUT.PUT_LINE('Payment failed: ' || SQLERRM);  
END;  
/
```

This case study links structure to a business rule. The rule is checked before the insert. The exception section prevents an unclear crash and provides a readable message.

Case Study 2: Bank withdrawal rule. A bank should stop a withdrawal when balance is not enough. This simple model shows the decision and the safe stop.

```
DECLARE
v_balance NUMBER := 2000;
v_withdraw NUMBER := 2500;
BEGIN
IF v_withdraw > v_balance THEN
RAISE_APPLICATION_ERROR(-20002, 'Insufficient balance. ');
END IF;

v_balance := v_balance - v_withdraw;
DBMS_OUTPUT.PUT_LINE('New balance: ' || v_balance);
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Transaction stopped: ' || SQLERRM);
END;
/
```

Case Study 3: Grade processing for many students. Many systems compute grades in a batch. The block below shows a cursor loop that updates grades based on marks. It also shows that one block can manage many rows in a controlled way.

```
DECLARE
CURSOR c_students IS
SELECT student_id, marks FROM result_table;
v_grade VARCHAR2(2);
BEGIN
FOR r IN c_students LOOP
IF r.marks >= 75 THEN
v_grade := 'A';
ELSIF r.marks >= 60 THEN
v_grade := 'B';
ELSIF r.marks >= 40 THEN
v_grade := 'C';
ELSE
v_grade := 'F';
END IF;
```

```
UPDATE result_table
SET grade = v_grade
WHERE student_id = r.student_id;

END LOOP;

DBMS_OUTPUT.PUT_LINE('Grades updated for all rows.');
```

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Grade update failed: ' || SQLERRM);

END;

/

In this case study, the block structure helps control a batch update. The DECLARE section prepares the cursor and a grade variable. The BEGIN section processes each row. The exception section protects the system if an unexpected issue occurs, such as a constraint problem.

Case Study 4: Library fine calculation with data lookup. A library system may compute a fine based on due date and return date. The block reads dates, computes days late, and calculates fine. If the issue record is missing, it handles the error and stops cleanly.

```
DECLARE
v_book_id NUMBER := 5001;
v_due_date DATE;
v_return_date DATE := SYSDATE;
v_days_late NUMBER;
v_fine NUMBER;
BEGIN
SELECT due_date INTO v_due_date
FROM library_issue
WHERE book_id = v_book_id;

v_days_late := v_return_date - v_due_date;

IF v_days_late <= 0 THEN
v_fine := 0;
ELSE
v_fine := v_days_late * 2;
END IF;

DBMS_OUTPUT.PUT_LINE('Fine for book ' || v_book_id || ' is ' || v_fine);
EXCEPTION
```

```
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('No issue record found for the book. ');
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Fine calculation failed: ' || SQLERRM);
END;
/
```

These examples show that the same block skeleton supports many domains. The main change is the business rule inside the BEGIN section and the expected exceptions inside the EXCEPTION section. Students should practice by rewriting one case study in their own words and then writing a new block for a different campus workflow, such as hostel billing or attendance warnings.

A PL/SQL block has a clear boundary and runs as one unit.

DECLARE is optional, but BEGIN and END are required.

Use := for assignment and use IF and LOOP in the executable part.

EXCEPTION handlers help give clear messages and protect data updates.

Anonymous blocks support practice, while named blocks support reuse and security.

Scope limits where variables can be used and reduces naming confusion.

Case studies show blocks enforce rules in fees, banking, grading, and libraries.

Careful transaction control avoids half-done updates in real systems.

5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple Meaning	Use In PL/SQL
Block	A complete mini program	Groups logic in one unit
Declarative part	Planning area	Declares variables and cursors
Executable part	Action area	Runs SQL and PL/SQL steps
Exception part	Safety area	Handles runtime errors
Anonymous block	Unnamed script	Quick testing and learning
Named block	Stored program	Procedures, functions, triggers
Scope	Visibility region	Controls variable access
Lifetime	Time of existence	Variable lasts during execution
%TYPE	Column-based type	Matches table column datatype
Cursor	Row pointer	Processes multiple rows
SELECT INTO	Fetch into variable	Reads data from a table
NO_DATA_FOUND	Missing row error	Raised when query finds none
TOO_MANY_ROWS	Extra row error	Raised when query finds many
WHEN OTHERS	General handler	Catches any exception

SQLERRM	Error text	Returns error message string
RAISE_APPLICATION_ERROR	Custom error	Enforces business rules

